



26

Using Animations

Animations can be used to improve the user experience in mobile applications. They usually help users to quickly recognize that something has changed, or help them focus on what is important. They improve the user experience and user satisfaction. Also, animations are simply fun to look at. For example, the heartbeat reaction in the Instagram app when you like a post or the Snapchat ghost animation when refreshing a page.

There are a couple of different approaches to processing and controlling animations in React Native. Firstly, we will take a look at animation tools that we can use, discover their pros and cons, and compare them. Then, we will implement several examples to get to know APIs better.

We'll cover the following topics in this chapter:

- Using React Native Reanimated
- Animating layout components
- Animating component styles

Technical requirements

You can find the code files for this chapter on GitHub at

<https://github.com/PacktPublishing/React-and-React-Native-5E/tree/main/Chapter26>.

Using React Native Reanimated

In the React Native world, we have a lot of libraries and approaches to animate our components, including the built-in **Animated API**. But in this chapter, I would like to opt for a library called **React Native Reanimated** and compare it with the Animated API to learn why it is the best choice.

The Animated API

The **Animated API** is the most common tool used to animate components in React Native. It has a set of methods that help you to create an animation object, control its state, and process it. The main benefit is that it can be used with any component, and not just animated components such as View or Text.

But, at the same time, this API has been implemented in the old architecture of React Native. Asynchronous communications between JavaScript and UI Native threads are used with the Animated API, delaying updates by at least one frame and lasting approximately 16 ms. Sometimes, the delay may last even longer if the JavaScript thread is running React's diff algorithm

and comparing or processing network requests simultaneously. The problem of dropped or delayed frames can be solved with the React Native Reanimated library, which is based on the new architecture and processes all business logic from the JavaScript thread in the UI thread.

React Native Reanimated

React Native Reanimated can be utilized to provide a more exhaustive abstraction of the Animated API to use with React Native. It provides an imperative API with multistage animations and custom transitions, while at the same time providing a declarative API that can be used to describe simple animations and transitions in a similar way to how CSS transitions work. It's built on top of React Native Animated and reimplements it on the Native thread. This allows you to use the familiar JavaScript language while taking advantage of the most high performance and simple API.

Furthermore, React Native Reanimated defines worklets, which are JavaScript functions that can be synchronously executed within the UI thread. This allows instant animations without having to wait for a new frame. Let's take a look at what a simple worklet looks like:

```
function simpleWorklet() {  
  "worklet";  
  console.log("Hello from UI thread");  
}
```

The only thing that is needed for the `simpleWorklet` function to get called inside the UI thread is to add the `worklet` directive at the top of the `function` block.

React Native Reanimated provides a variety of hooks and methods that help us handle animations:

- `useSharedValue`: This hook returns a `SharedValue` instance, which is the main stateful data object that lives in the UI thread context and has a similar concept to `Animated.Value` in the core Animated API. A Reanimated animation is triggered when `SharedValue` is changed. The key benefit is that updates to shared values can be synchronized across the React Native and UI threads without triggering a re-render. This enables complex animations to run smoothly at 60 FPS without blocking the JS thread.
- `useDerivedValue`: This hook creates a new shared value that automatically updates whenever the shared values used in its calculation change. It allows you to create shared values that depend on other shared values, while keeping them all reactive. `useDerivedValue` is used to create a **de-**

rived state in a worklet that runs on the UI thread based on updates to the source shared values. This derived state can then drive animations or other side effects without triggering a re-render on the JS thread.

- `useAnimatedStyle` : The hook allows you to create a style object with the ability to animate its properties based on shared values. It maps shared value updates to the corresponding view properties. `useAnimatedStyle` is the main way to connect shared values to views and enable smooth animations running on the UI thread.
- `withTiming`, `withSpring`, `withDecay` : These are animation utility methods that update a shared value in a smooth, animated way using various curves and physics. They allow you to define animations declaratively by specifying the target value and animation configuration.

We have learned what React Native Reanimated is and how it is different from the Animated API. Next, let's try to install it and apply it to our app.

Installing the React Native Reanimated library

To install the React Native Reanimated library, run this command inside your Expo project:

```
expo install react-native-reanimated
```

After the installation is complete, we need to add the **Babel** plugin to `babel.config.js`:

```
module.exports = function(api) {  
  api.cache(true);  
  return {  
    presets: ['babel-preset-expo'],  
    plugins: ['react-native-reanimated/plugin'],  
  };  
};
```

The main purpose of that plugin is to convert our JavaScript worklet functions into functions that will work in the UI thread.

After you add the Babel plugin, restart your development server and clear the bundler cache:

```
expo start --clear
```

This section has introduced us to the React Native Reanimated library. We have found out why it is better than the built-in Animated API. In the next sections, we will use it in real examples.

Animating layout components

A common use case is animating the entering and exiting layouts of your components. This means that when your component renders for the first time and when you unmount your component, it appears animated. React Native Reanimated is an API that lets you animate layouts and add animations such as `FadeIn`, `BounceIn`, and `ZoomIn`.

React Native Reanimated also provides a special `Animated` component that is the same as the `Animated` component in the `Animated` API, but with additional props:

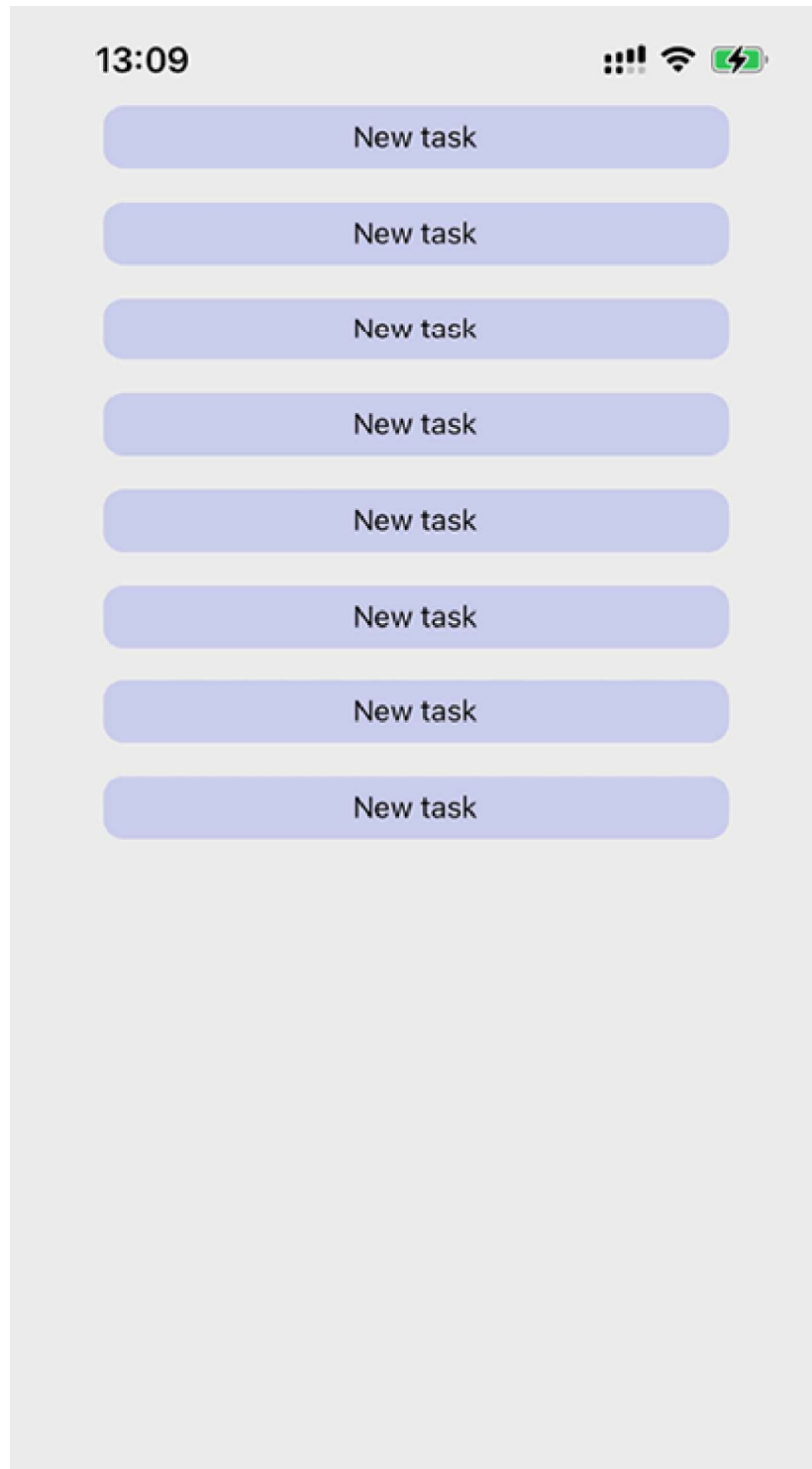
- `entering`: Accepts a predefined animation when the component mounts and renders
- `exiting`: Accepts the same animation object, but it will be called when the component unmounts

Let's create a simple to-do list with a button for creating tasks and a feature that allows us to delete tasks when we click on them.



It's impossible to see animations in screenshots, so I suggest you open the code and try to implement the animations to see the results.

Firstly, let's take a look at the main screen of our to-do list app and how the items are rendering at the moment:



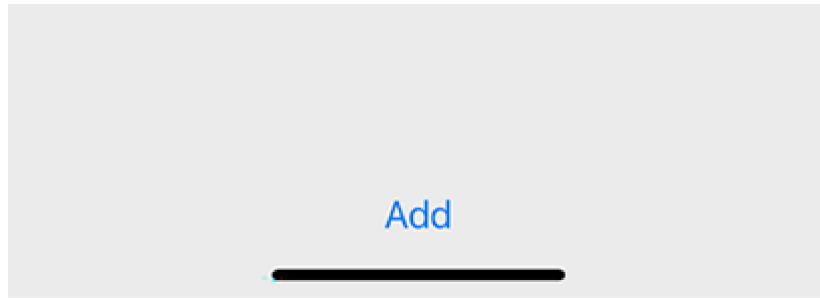


Figure 26.1: To-do list

This is a simple example with a list of task items and one button for adding new tasks. When we quickly press the **Add** button several times, the list items come from the left side of the screen with an animation:



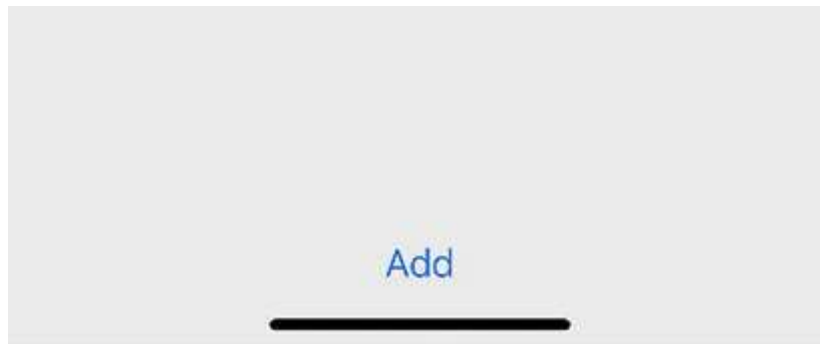


Figure 26.2: To-do list with animated rendering

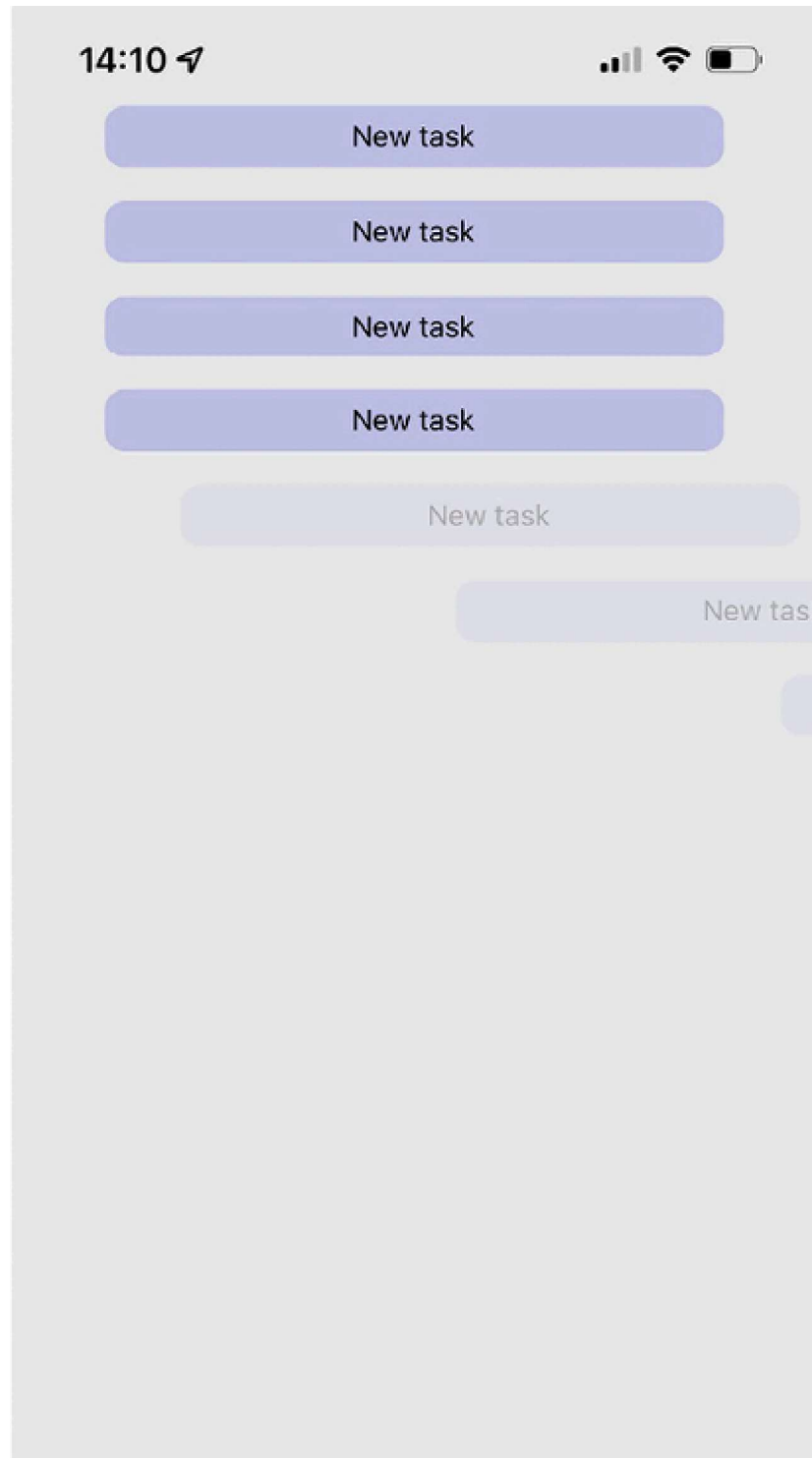
The magic is implemented in the `TodoItem` component. Let's take a look at it:

```
export const TodoItem = ({ id, title, onPress }) => {  
  return (  
    <Animated.View entering={SlideInLeft}  
      exiting={SlideOutRight}>  
      <TouchableOpacity onPress={() => onPress(id)}  
        style={styles.todoItem}>  
        <Text>{title}</Text>  
      </TouchableOpacity>  
    </Animated.View>  
  );  
};
```

As you can see, there is no complicated logic, and there isn't too much code. We just take the `Animated` component as the

root of animation and pass predefined animations from the React Native Reanimated library to the `entering` and `exit-ing` props.

To see how the items disappear from the screen, we need to press the to-do items so the exiting animation will run. I've pressed a few items and tried to catch the result in the following screenshot:



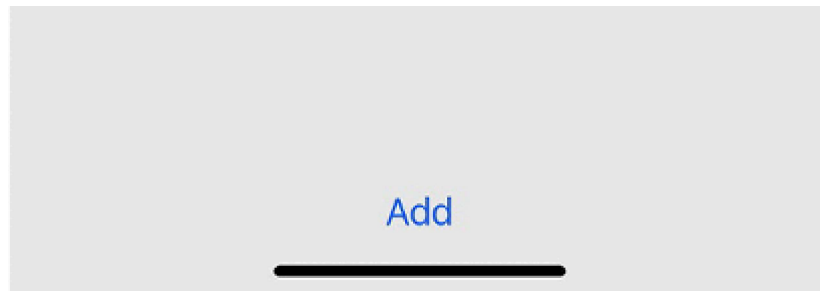


Figure 26.3: Deleting to-do items from the screen

Let's examine the `App` component to see the entire picture:

```
export default function App() {  
  const [todoList, setTodoList] = useState([]);  
  const addTask = () => {  
    setTodoList([  
      ...todoList,  
      { id: String(new Date().getTime()), title: "New task"  
    },  
    ]);  
  };  
  const deleteTask = (id) => {  
    setTodoList(todoList.filter((todo) => todo.id !== id));  
  };  
}
```

We have created a `todoList` state using the `useState` hook and handler functions for adding and deleting tasks. Next, let's take a look at how the animation will be applied to the layout:

```
return (  
  <View style={styles.container}>  
    <View style={{ flex: 1 }}>  
      {todoList.map(({ id, title }) => (  
        <TodoItem key={id} id={id} title={title}  
          onPress={deleteTask} />  
      ))}  
    </View>  
    <Button onPress={addTask} title="Add" />  
  </View>  
);  
}
```

In this example, we learned a simple way to apply animations to make our app look better. However, the React Native Reanimated library is a lot more powerful than we imagined. The next example illustrates how we can animate and create our own animations by applying them directly to the styles of our components.

Animating component styles

In a more complex example, I suggest creating a button with beautiful tappable feedback. This button will be built using the `Pressable` component that we learned about in *Chapter 23*, *Responding to User Gestures*. This component accepts the `onPressIn`, `onLongPress`, and `onPressOut` events. As a result of

these events, we will be able to see how our touches will be reflected on the button.

Let's start by defining `SharedValue` and `AnimatedStyle`:

```
const radius = useSharedValue(30);
const opacity = useSharedValue(1);
const scale = useSharedValue(1);
const color = useSharedValue(0);
const backgroundColor = useDerivedValue(() => {
  return interpolateColor(color.value, [0, 1], ["orange", "red"]);
});
const animatedStyles = useAnimatedStyle(() => {
  return {
    opacity: opacity.value,
    borderRadius: radius.value,
    transform: [{ scale: scale.value }],
    backgroundColor: backgroundColor.value,
  };
}, []);
```

In order to animate style properties, we have created a `SharedValue` object using the `useSharedValue` hook. It takes default values as an argument. Next, we created the style object with the `useAnimatedStyle` hook. The hook accepts the callback that should return a style object. The `useAnimatedStyle` hook is similar to the `useMemo` hook, but all calculations are performed in the UI thread and all `SharedValue` changes

will invoke the hook to recalculate the style object. The background color of the button was created using `useDerivedValue` by interpolating between orange and red to provide a smooth transition.

Next, let's create handler functions that will update the style properties in relation to the pressing state of the button:

```
const onPressIn = () => {
  radius.value = withSpring(20);
  opacity.value = withSpring(0.7);
  scale.value = withSpring(0.9);
};
const onLongPress = () => {
  scale.value = withSpring(0.8);
  color.value = withSpring(1);
};
const onPressOut = () => {
  radius.value = withSpring(30);
  opacity.value = withSpring(1);
  scale.value = withSpring(1, { damping: 50 });
  color.value = withSpring(0);
};
```

The first handler, `onPressIn`, updates `borderRadius`, `opacity`, and `scale` from their default values. We also update these values using `withSpring`, which makes updating styles smoother. Like the first handler, other ones will also update

the style of the button but in different ways. `onLongPress` turns the button red and makes it smaller. `onPressOut` resets all values to their default values.

We've implemented all necessary logic and can now apply it to the layout:

```
<View style={styles.container}>
  <Animated.View style={[styles.buttonContainer,
    animatedStyles]}>
    <Pressable
      onPressIn={onPressIn}
      onPressOut={onPressOut}
      onLongPress={onLongPress}
      style={styles.button}
    >
      <Text style={styles.buttonText}>Press me</Text>
    </Pressable>
  </Animated.View>
</View>
```

Finally, let's take a look at the result:



Figure 26.4: Button with default, pressed, and long-pressed styles

In *Figure 26.4*, you can see the three states of the button: default, pressed, and long-pressed.

Summary

In this chapter, we've learned how to use the React Native Reanimated library to add animations to the layout and components. We've gone through the basic principles of the library and found out how it works under the hood and how it executes code inside the UI thread without using Bridge to connect JavaScript and Native layers of the app.

We also went through two examples using the React Native Reanimated library. In the first example, we learned how to apply a layout animation using predefined declarative animations to get our component to appear and disappear beautifully. In the second example, we animated the button's styles with the `useSharedValue` and `useAnimatedStyle` hooks.

Skills to animate components and layout will help you make your app more beautiful and responsive. In the next chapter, we'll learn about controlling images in our apps.

